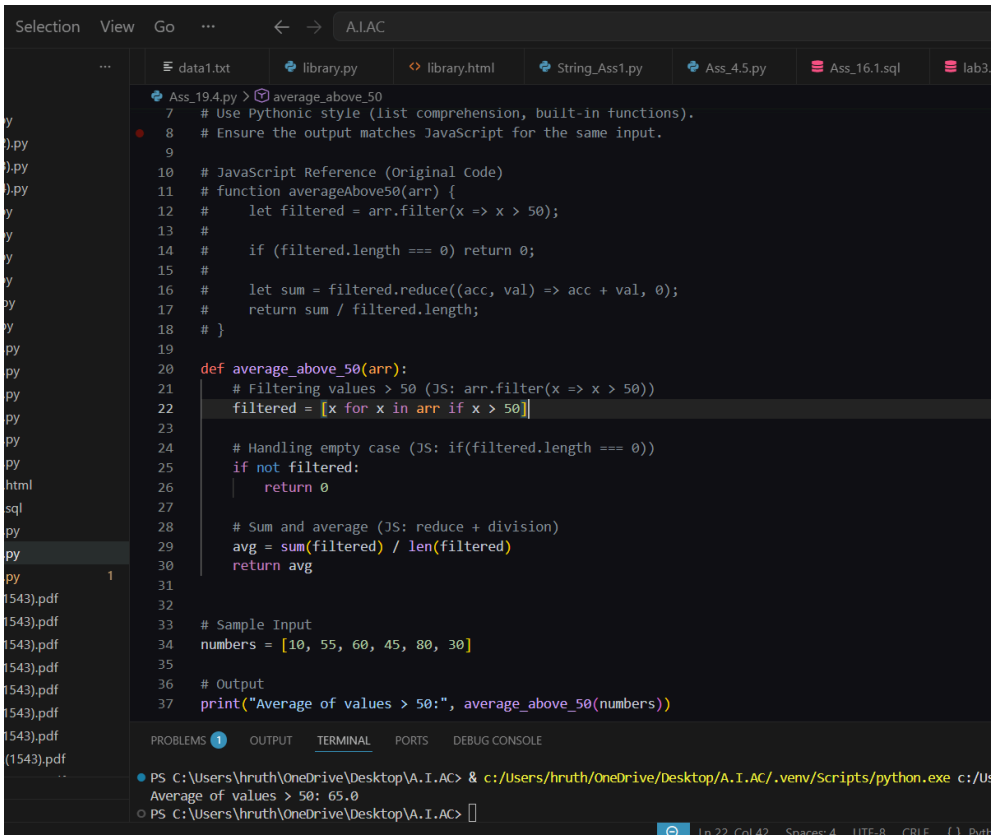
AI Assisted Coding Assignment-19.4

> 2303A51543 BT:29 VELDI.HRUTHIKA 16-4-26

💡 Task 1 – JavaScript to Python (Array Processing Logic)
Scenario
A frontend application written in JavaScript processes user scores. The backend team wants the same logic implemented in Python for analytics processing.
Task
Translate a JavaScript function that:
• Takes an array of numbers
• Filters values greater than 50
• Returns the average of filtered values
Instructions
• Prompt AI to convert JavaScript array methods (filter, reduce) into equivalent Python logic.
• Ensure list comprehension or Pythonic style is used.
• Compare outputs of both implementations with sample input.
• Document differences in handling arrays between JS and Python.
Expected Output
• Equivalent Python function.
• Same output for identical inputs.
• Clean and Pythonic implementation.

```python
#2303a51543
# Task 1 – JavaScript to Python (Array Processing Logic)
# Convert the following JavaScript logic into Python:
# Take an array of numbers
# Filter values greater than 50
# Return the average of filtered values
# Use Pythonic style (list comprehension, built-in functions).
# Ensure the output matches JavaScript for the same input.

# JavaScript Reference (Original Code)
# function averageAbove50(arr) {
#     let filtered = arr.filter(x => x > 50);
#
#     if (filtered.length === 0) return 0;
#
#     let sum = filtered.reduce((acc, val) => acc + val, 0);
#     return sum / filtered.length;
# }

def average_above_50(arr):
    # Filtering values > 50 (JS: arr.filter(x => x > 50))
    filtered = [x for x in arr if x > 50]
```

```python
    # Handling empty case (JS: if(filtered.length === 0))
    if not filtered:
        return 0

    # Sum and average (JS: reduce + division)
    avg = sum(filtered) / len(filtered)
    return avg


# Sample Input
numbers = [10, 55, 60, 45, 80, 30]

# Output
print("Average of values > 50:", average_above_50(numbers))
```



💡 Task 2 – Python OOP to C++ Class Translation
Scenario
A data modeling module is written in Python using OOP. The system is being migrated to a high-performance C++ environment.
Task
Translate a Python class that:
• Contains private attributes
• Uses getter/setter methods
• Includes a method that calculates a derived value
Instructions
• Convert the Python class into a C++ class.
• Ensure:
o Proper access modifiers
o Constructor handling
o Correct data types
• Compile and test the C++ version.
• Compare how encapsulation differs between Python and C++.
Expected Output
• Fully working C++ class equivalent to Python version.
• Proper use of access specifiers (private, public).

Convert the following Python OOP class into an equivalent C++ class.
// Requirements:
// - Use private attributes
// - Implement getter and setter methods
// - Include a method to calculate a derived value
// - Use proper access specifiers (private, public)
// - Add constructor for initialization
// - Ensure correct data types and clean structure

#include <iostream>
using namespace std;

// Python Reference (Original Code)
/*
class Student:
def **init**(self, marks):
self.__marks = marks

```
def get_marks(self):
    return self.__marks

def set_marks(self, marks):
    self.__marks = marks

def calculate_grade(self):
    if self.__marks >= 50:
        return "Pass"
    else:
        return "Fail"
```

- /

class Student {
private:
int marks; // private attribute

public:
// Constructor
Student(int m) {
marks = m;
}

```
// Getter
int getMarks() {
    return marks;
}

// Setter
void setMarks(int m) {
    marks = m;
}

// Method to calculate derived value
string calculateGrade() {
    if (marks >= 50)
        return "Pass";
    else
        return "Fail";
}
```

};

int main() {
Student s(60);

```
cout << "Marks: " << s.getMarks() << endl;
cout << "Grade: " << s.calculateGrade() << endl;

s.setMarks(40);
cout << "Updated Marks: " << s.getMarks() << endl;
cout << "Updated Grade: " << s.calculateGrade() << endl;

return 0;
```

}

```
# Output
print("Average of values > 50:", average_above_50(numbers))

#include <iostream>
using namespace std;

class AverageCalculator {
private:
    int* arr;
    int size;

public:
    // Constructor
    AverageCalculator(int* input_arr, int input_size) {
        size = input_size;
        arr = new int[size];
        for (int i = 0; i < size; i++) {
            arr[i] = input_arr[i];
        }
    }

    // Destructor
    ~AverageCalculator() {
        delete[] arr;
    }

    // Getter for array
    int* getArray() const {
        return arr;
    }

    // Getter for size
    int getSize() const {
        return size;
    }

    // Calculate average of values > 50
    double calculateAverageAbove50() const {
```

```
PS C:\Users\hruth\OneDrive\Desktop\A.I.AC> & c:/Users/hruth/OneDrive/Desktop/A.I.AC/.venv/Scripts/python.exe c:/Users/hruth/OneDrive/Desktop/A.I.AC/Ass_19.4.py
Average of values > 50: 65.0
```

https://www.onlinegdb.com

GDB online Debugger | Compiler - Code, Compile, Run, Debug online C, C++

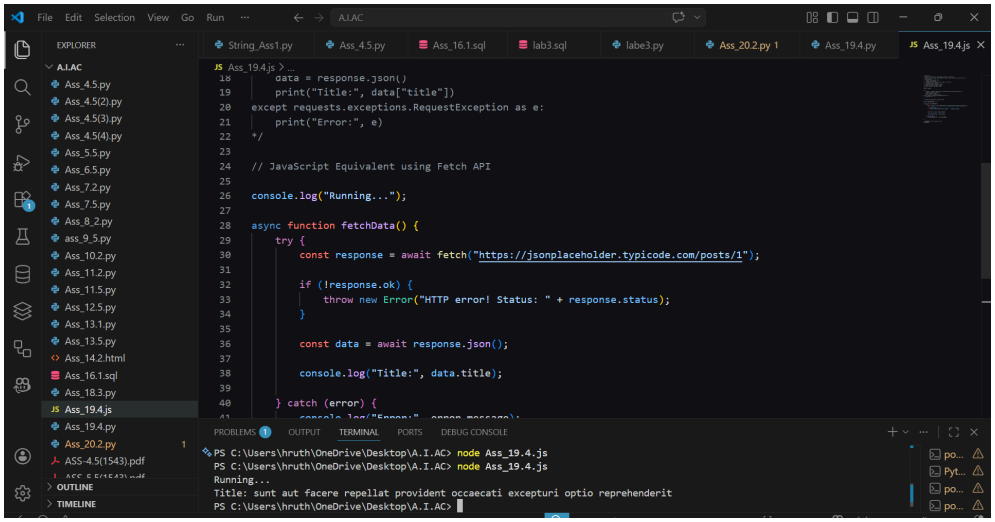
```
        return "Fail"
*/

class Student {
private:
    int marks;   // private attribute

public:
    // Constructor
    Student(int m) {
        marks = m;
    }

    // Getter
    int getMarks() {
        return marks;
    }

    // Setter
    void setMarks(int m) {
        marks = m;
    }

    // Method to calculate derived value
    string calculateGrade() {
        if (marks >= 50)
            return "Pass";
        else
            return "Fail";
```

```
Marks: 60
Grade: Pass
Updated Marks: 40
Updated Grade: Fail
```

Task 3 – REST API Call Translation: Python to JavaScript
Scenario
A backend developer wrote an API request using Python requests. The frontend team needs the same functionality in JavaScript (Fetch API).
Task
Translate Python API request code into JavaScript.
Instructions
• Convert:
o GET request logic
o JSON parsing
o Error handling (try-except → try-catch)
• Ensure proper async/await usage in JavaScript.
• Compare differences in error handling mechanisms.
Expected Output
• Equivalent JavaScript code using Fetch API.
• Proper async handling.
• Similar output structure.

```
//2303a51543
// Task 3 – REST API Call Translation: Python to JavaScript
// Convert the following Python REST API request into JavaScript using Fetch API.
// Requirements:
// - Convert GET request logic
// - Handle JSON parsing
// - Implement error handling (try-except → try-catch)
// - Use async/await properly
// - Maintain similar output structure
// Python Reference (Original Code)
// Python Reference (Original Code)
/*
import requests

try:
    response = requests.get("https://jsonplaceholder.typicode.com/posts/1")
    response.raise_for_status()
    data = response.json()
    print("Title:", data["title"])
except requests.exceptions.RequestException as e:
    print("Error:", e)
*/

// JavaScript Equivalent using Fetch API

console.log("Running...");

async function fetchData() {
    try {
        const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
```

```
        if (!response.ok) {
            throw new Error("HTTP error! Status: " + response.status);
        }

        const data = await response.json();

        console.log("Title:", data.title);

    } catch (error) {
        console.log("Error:", error.message);
    }
}

// IMPORTANT — this actually runs it
fetchData();
```



💡 Task 4 – SQL Aggregation to MongoDB Query Translation
Scenario
A company is migrating from relational databases (SQL) to MongoDB (NoSQL).
Task
Translate an SQL aggregation query into a MongoDB aggregation pipeline.
Example Logic:
• Group records by category
• Calculate average price
• Filter groups where average price > threshold
Instructions
• Provide SQL query to AI.
• Generate equivalent MongoDB query.
• Explain differences between relational and document-based querying.
• Validate results conceptually.
Expected Output
• Working MongoDB aggregation query.
• Clear understanding of grouping and aggregation differences.

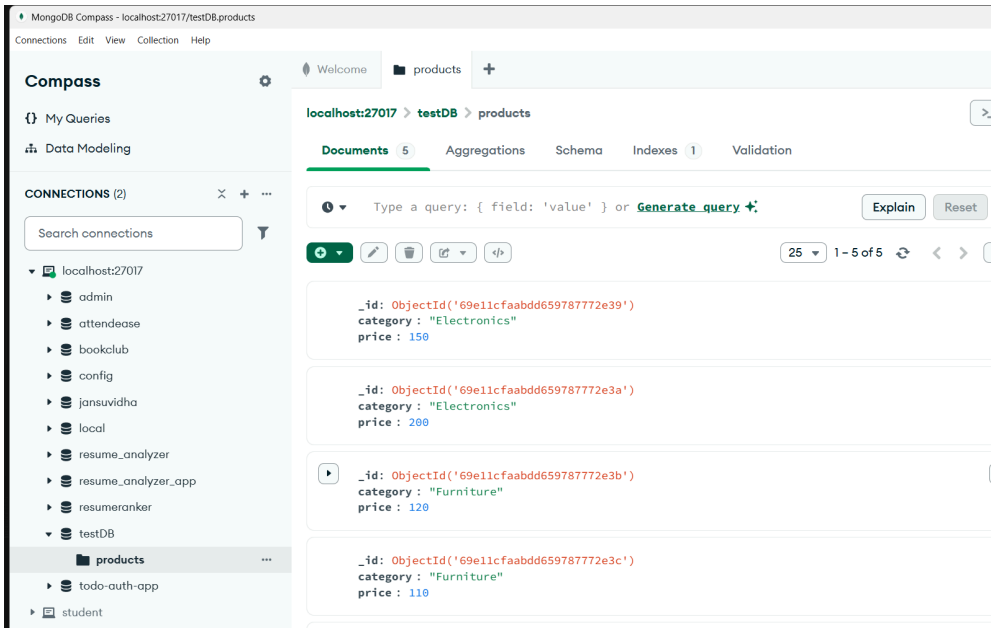
```
-- Prompt:
-- Convert the following SQL aggregation query into MongoDB aggregation pipeline.
-- Requirements:
-- - Group by category
-- - Calculate average price
-- - Filter groups where average price > 100
-- - Maintain equivalent logic and output structure
```

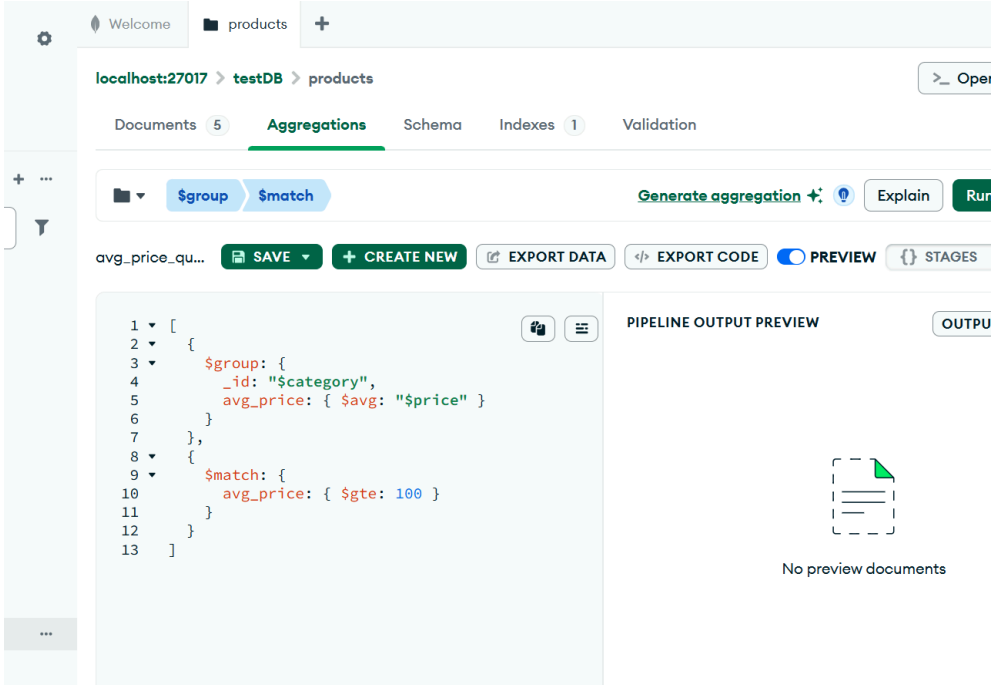
1. SQL Query

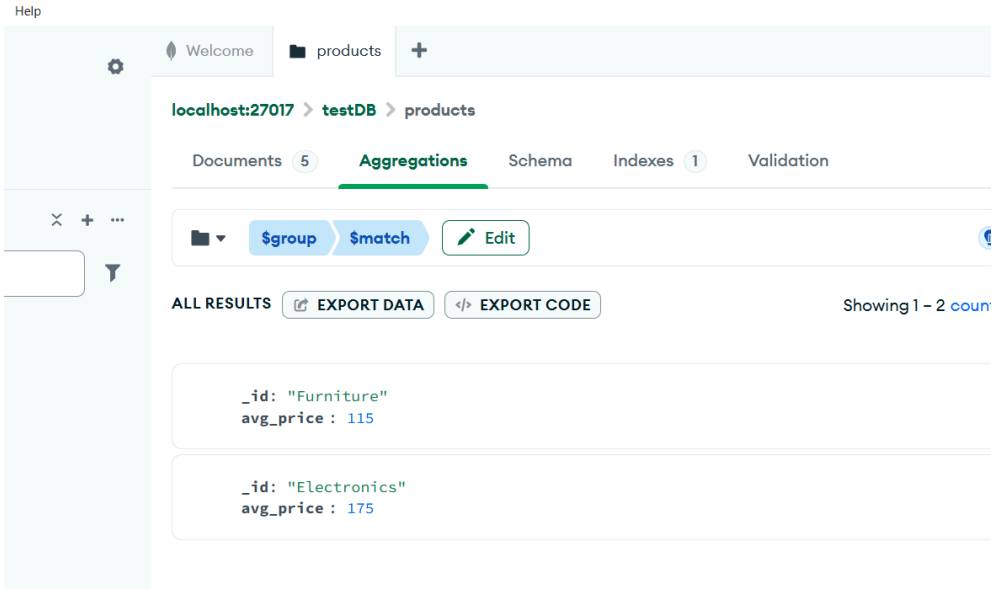
```
SELECT category, AVG(price)AS avg_price
FROM products
GROUPBY category
HAVING AVG(price)>100;
```

2. MongoDB Query

```
db.products.aggregate([
  {
    $group: {
      _id:"$category",
      avg_price: { $avg:"$price" }
    }
  },
  {
    $match: {
      avg_price: { $gt:100 }
    }
  }
]);
```


MongoDB Compass - localhost:27017/testDB.products
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (2)
Search connections
localhost:27017
admin
attendease
bookclub
config
jansuvidha
local
resume_analyzer
resume_analyzer_app
resumeranker
testDB
products
todo-auth-app
student
Welcome
products
localhost:27017 > testDB > products
Documents 5
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
25
1 - 5 of 5
_id: ObjectId('69e11cfaabdd659787772e39')
category : "Electronics"
price : 150
_id: ObjectId('69e11cfaabdd659787772e3a')
category : "Electronics"
price : 200
_id: ObjectId('69e11cfaabdd659787772e3b')
category : "Furniture"
price : 120
_id: ObjectId('69e11cfaabdd659787772e3c')
category : "Furniture"
price : 110


Welcome
products
localhost:27017 > testDB > products
Documents 5
Aggregations
Schema
Indexes 1
Validation
$group
$match
Generate aggregation
Explain
avg_price_qu...
SAVE
CREATE NEW
EXPORT DATA
EXPORT CODE
PREVIEW
STAGES
[
{
$group: {
_id: "$category",
avg_price: { $avg: "$price" }
}
},
{
$match: {
avg_price: { $gte: 100 }
}
}
]
PIPELINE OUTPUT PREVIEW
No preview documents



Task 5 – Real-Time Application: Algorithm Translation Across Three Languages

Scenario

A tech company maintains codebases in multiple languages (Python, Java, and Go). A common algorithm must be implemented consistently across systems.

Task

Choose an algorithm (e.g., binary search, palindrome check, prime number checker) and translate it across:

- Python
- Java
- One additional language (Go / C# / JavaScript)

Instructions

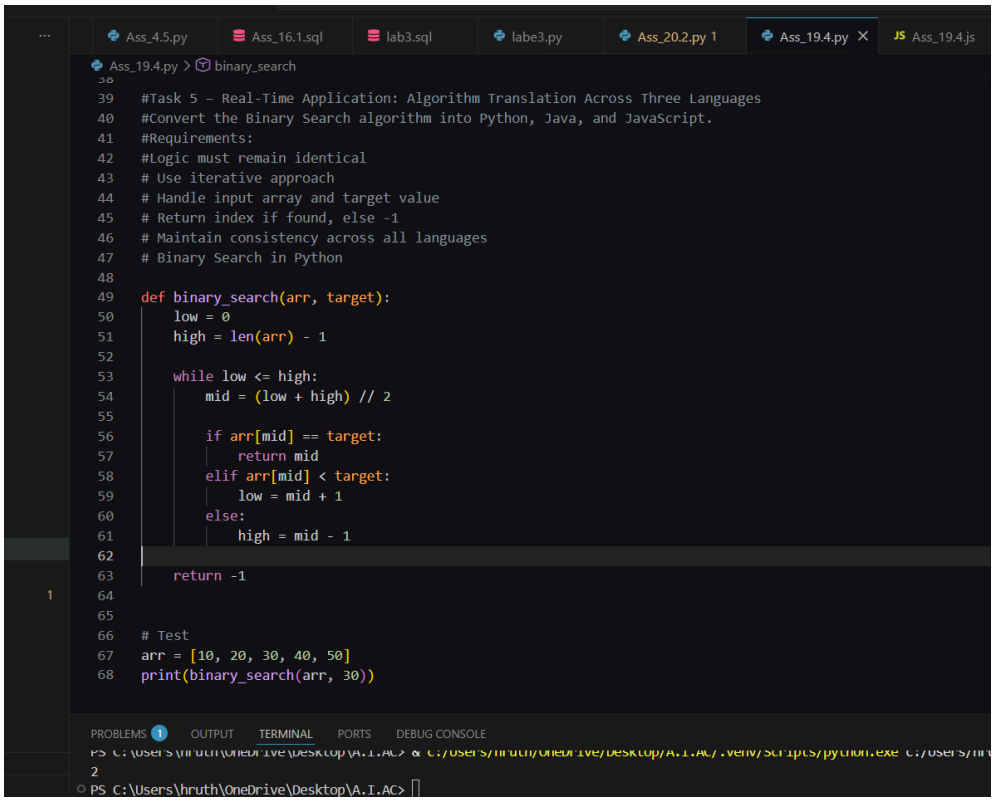
- Ensure logic remains identical across languages.
- Handle:
  - o Data types
  - o Loop differences
  - o Input/output handling
- Test each version with same input values.
- Briefly document translation challenges.

Expected Output

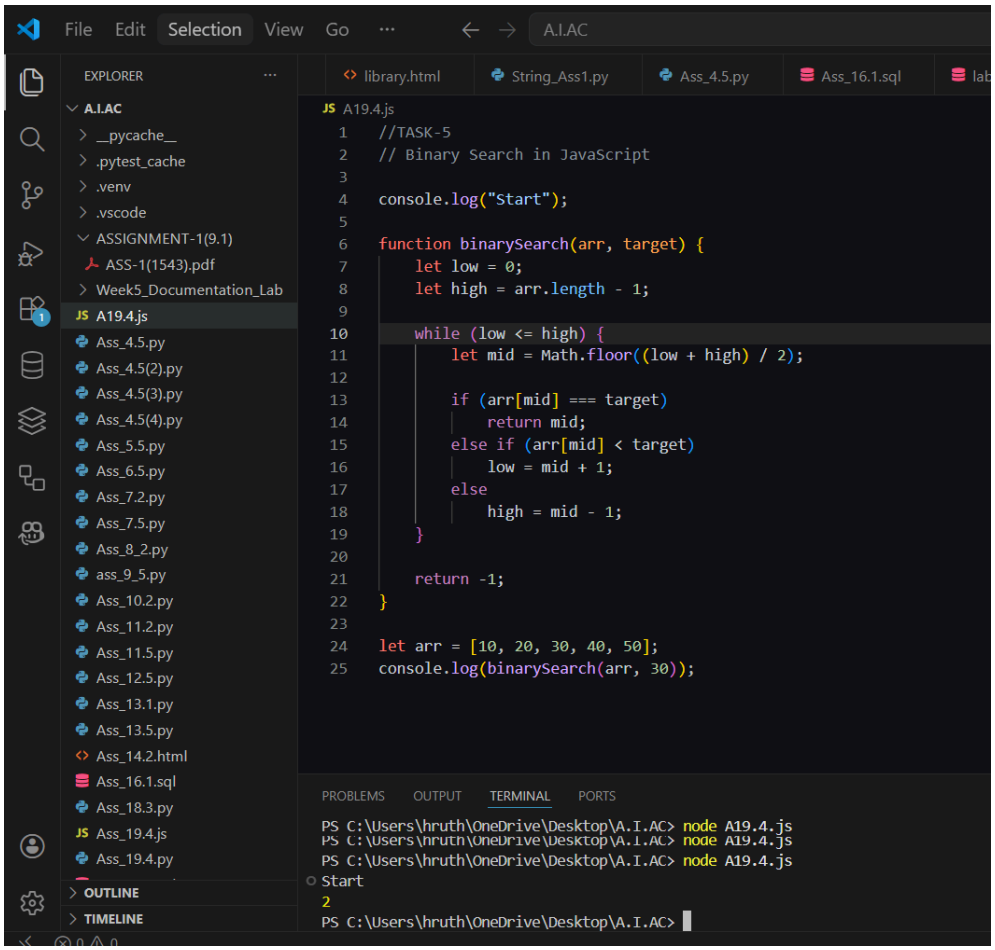
- Three equivalent working implementations.
- Clear logical consistency.
- Understanding of syntactic vs structural differences.

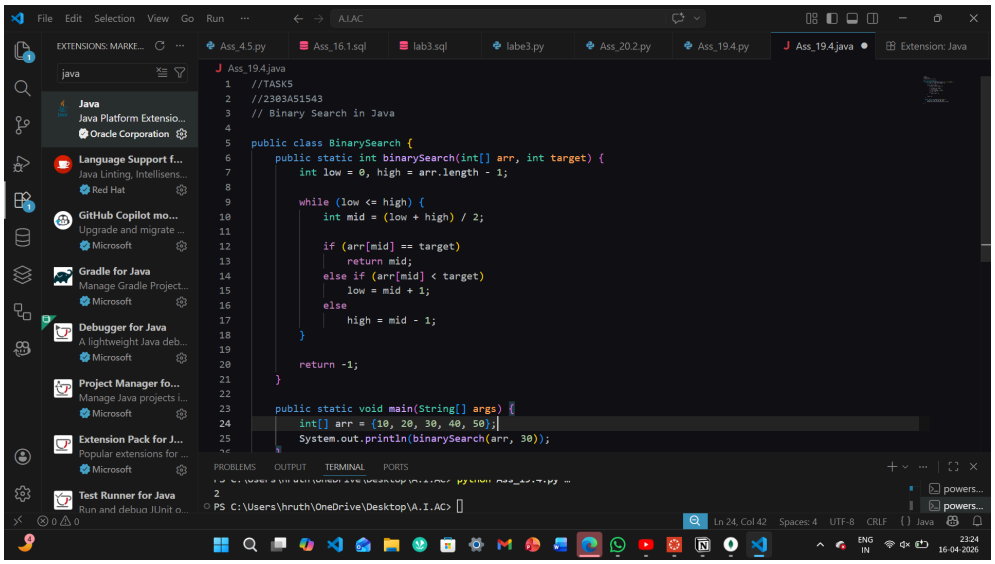
```
#Task 5 – Real-Time Application: Algorithm Translation Across Three Languages
#Convert the Binary Search algorithm into Python, Java, and JavaScript.
#Requirements:
#Logic must remain identical
# Use iterative approach
# Handle input array and target value
# Return index if found, else -1
```

```python
# Maintain consistency across all languages
# Binary Search in Python

def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1


# Test
arr = [10, 20, 30, 40, 50]
print(binary_search(arr, 30))
```



```javascript
//TASK-5
// Binary Search in JavaScript

console.log("Start");

function binarySearch(arr, target) {
    let low = 0;
    let high = arr.length - 1;

    while (low <= high) {
        let mid = Math.floor((low + high) / 2);

        if (arr[mid] === target)
            return mid;
        else if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}

let arr = [10, 20, 30, 40, 50];
console.log(binarySearch(arr, 30));
```




```java
//TASK5
//2303A51543
// Binary Search in Java

public class BinarySearch {
    public static int binarySearch(int[] arr, int target) {
        int low = 0, high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (arr[mid] == target)
                return mid;
            else if (arr[mid] < target)
                low = mid + 1;
            else
                high = mid - 1;
        }

        return -1;
    }
```

```java
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(arr, 30));
    }
}
```